

CSCI 135 Assignment – Exceptional Fractions

November 1st / 3rd , 2023

Due Date: Sunday, 11/5/2023 11:59PM **(THIS IS A BIG LAB. START EARLY!)**

Points: 100

Topic: Functions, Exception Handling and Error Checking

In this assignment, you will be writing solid, well-tested functions that perform correctly for expected inputs and that raise exceptions for problematic inputs in a controlled manner. You will write code that represents fractions as tuples of two integers (numerator and denominator) and manipulate these fractions using standard operations.

I am providing you with a starter `ExceptionalFractions.py` file that implements several of the necessary functions and provides headers describing the remaining functions. It is your responsibility to write the code for the remaining functions. I am also providing you with a starter `TestExceptionalFractions.py` file that tests the functions from `ExceptionalFractions.py`. You should add code to this file to test all the functions thoroughly, particularly those you implement. You should write test code not only to show that the functions perform correctly for expected inputs and that they raise exceptions for problematic inputs.

See attached starter files: `ExceptionalFractions.py`, `TestExceptionalFractions.py`

Use these as a starting point. Make sure you add/implement all requested items.

Submission:

After following all instructions, please submit the Python scripts in Moodle. Upload these as individual files – do not zip them. Be sure the files have an accurate header section including the file name, your name, a description of the program, and other useful information such as example usage.

- `ExceptionalFractions.py`
- `TestExceptionalFractions.py`

CSCI 135 Assignment – Exceptional Fractions

Grading:

The programs/scripts are worth 100 points. You will be graded according to the following criteria:

	Points
Header Comments for Entire Scripts	5
Descriptive Comments Throughout Code	10
Header Comments for Functions	5
<code>gcd()</code> works and written correctly	5
<code>makeFraction()</code> works and written correctly	5
<code>isValidFraction()</code> works and written correctly	5
<code>fractionToString()</code> works and written correctly	5
<code>fractionToDecimal()</code> works and written correctly	5
<code>reduceFraction()</code> works and written correctly	5
<code>invertFraction()</code> works and written correctly	5
<code>negateFraction()</code> works and written correctly	5
<code>areFractionsEquivalent()</code> works and written correctly	5
<code>multiplyFractions()</code> works and written correctly	5
<code>divideFractions()</code> works and written correctly	5
<code>addFractions()</code> works and written correctly	5
<code>subtractFractions()</code> works and written correctly	5
<code>TestExceptionalFractions.py</code>	20
Total	100

ExceptionalFractions.py

This file includes functions that implement the functionality for fractions as described. Pay careful attention to the functions I provide in the starter file as well as the headers for functions that need to be implemented.

The functions, including some that I have provided are:

- ✓ `gcd(n1, n2)`
- ✓ `makeFraction(numerator, denominator)`
- ✓ `isValidFraction(f)`
- `fractionToString(f)`
- `fractionToDecimal(f)`
- ✓ `reduceFraction(f)`
- `invertFraction(f)`
- `negateFraction(f)`
- `areFractionsEquivalent(f, g)`
- ✓ `multiplyFractions(f, g)`
- `divideFractions(f, g)`
- `addFractions(f, g)`
- `subtractFractions(f, g)`

`f` and `g` are expected to be tuples representing fractions

`n1` and `n2` are expected to be integers

`numerator` and `denominator` are expected to be integers

Here are several examples of tuple representations of fractions:

`(1, 2)` represents $\frac{1}{2}$

`(4, 3)` represents $\frac{4}{3}$

`(0, 1)` represents $\frac{0}{1}$, which is equivalent to 0.

`(-1, 3)` represents $-\frac{1}{3}$

`(10, 0)` is not allowed because it is not a valid fraction – it involves division by zero

Tuples that don't contain two integers are not allowed, and types other than tuples aren't allowed as the container. Attempts to use invalid representations should result in exceptions being thrown.

CSCI 135 Assignment – Exceptional Fractions

Example operations include:

- `gcd(100,64)` # returns 4
- `makeFraction(1,2)` # returns (1, 2)
- `isValidFraction((1,2))` # returns True
- `isValidFraction((1,0))` # returns False
- `fractionToString((1,2))` # returns '1/2'
- `fractionToDecimal((1,2))` # returns 0.5
- `reduceFraction((6,-8))` # returns (-3, 4)
- `invertFraction((1,2))` # returns (2, 1)
- `negateFraction((1,2))` # returns (-1, 2)
- `areFractionsEquivalent((1,2),(3,6))` # returns True
- `multiplyFractions((1,2),(2,3))` # returns (1, 3)
- `divideFractions((1,2),(1,4))` # returns (2, 1)
- `addFractions((1,2),(1,4))` # returns (3, 4)
- `subtractFractions((1,2),(1,3))` # returns (1, 6)

TestExceptionalFractions.py

This file includes code to test the functionality for fractions as described. Pay careful attention to the test code I provide in the starter file as well as placeholders for functions that need to be tested. NOTE: I do not tell you how or how much testing should be done for each. Use the examples I provided as a guide while being thorough with your testing. You should test with representative/expected inputs and also with problematic inputs that should raise exceptions. Note how I handle cases in which there are exceptions with try-except blocks. Without this, the test script would crash on the first exception.

You should challenge yourself to thoroughly test everything in your suite of tests, in anticipation of the tests we will run when grading your submission. Make a goal to ensure that your code works correctly!

Continue making your code organized and readable. Pay attention to your header comments and include meaningful comments in-line with your code. Use proper indentation and understandable variable names. Use whitespace to help with readability and logical grouping of sections of code.

You are writing reusable, solid code for someone else.

Helpful hints:

Use the starter files that I provided. Read them, run them, use them as a guide.

Work incrementally. Build and test one function at a time, starting with the easier ones.

Recall how fraction operations work. Addition might be the trickiest of these ...

Use the functions you write as helpers in other functions you have to write. For instance, subtraction can be turned into an addition problem with the right helper functions. Similarly, division can be turned into multiplication. This might explain why I included certain prior functions.

Don't just check for proper operation. Check for invalid inputs and other things that could go wrong. Try to break your code and implement fixes so it won't break when we test it. Generating known exceptions for invalid inputs is part of this.

Upload ExceptionalFractions.py and TestExceptionalFractions.py to Moodle as your submission for this assignment.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for the content within, so you really should work through them ...

From me:

1. Look ahead in our book(s) at Objects and Classes, Modules and Packages, etc. Start thinking about how the functionality for fractions that we cover in this assignment could be better packaged for use. Hint: We will do some of this in the near future.